In this notebook, I work into a comprehensive exploratory data analysis(EDA) of Amazon Prime Video datasets. My objectives are to allow me to extract valuable insights such as:

- Content Diversity: What genres and categories dominate the platform?
- Regional Availability: How does content distribution vary across different regions?
- Trends Over Time: How has Amazon Prime's content library evolved?
- IMDb Ratings & Popularity: What are the highest-rated or most popular shows on the platform?

SECTION 1:

```
- IMPORTING  DATASET LIBARARIES
- CLEANING THE DATA:
- EDA to understand patterns
```

In [50]:

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
!pip install wordcloud matplotlib
from wordcloud import WordCloud
import warnings
warnings.filterwarnings('ignore')
```

```
Requirement already satisfied: wordcloud in c:\users\gaire\anaconda3\li
b\site-packages (1.9.6)
Requirement already satisfied: matplotlib in c:\users\gaire\anaconda3\l
ib\site-packages (3.10.6)
Requirement already satisfied: numpy>=1.19.3 in c:\users\gaire\anaconda
3\lib\site-packages (from wordcloud) (2.3.5)
Requirement already satisfied: pillow in c:\users\gaire\anaconda3\lib\s
ite-packages (from wordcloud) (12.0.0)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\gaire\anaco
nda3\lib\site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in c:\users\gaire\anaconda3
\lib\site-packages (from matplotlib) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\gaire\anac
onda3\lib\site-packages (from matplotlib) (4.60.1)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\gaire\anac
onda3\lib\site-packages (from matplotlib) (1.4.9)
Requirement already satisfied: packaging>=20.0 in c:\users\gaire\anacon
da3\lib\site-packages (from matplotlib) (25.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\gaire\anaco
nda3\lib\site-packages (from matplotlib) (3.2.5)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\gaire\a
naconda3\lib\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in c:\users\gaire\anaconda3\lib
\site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
```

In [51]:

```python
# Since we have two datasets. I have created df1 and df2.
df1= pd.read_csv(r"C:\Users\gaire\Downloads\Amazon_Prime_Titles.csv")
df2= pd.read_csv(r"C:\Users\gaire\Downloads\Amazon_Prime_Casting_Credi
```

In [52]:

```python
df1.head(2)
```

Out[52]:

	id	title	type	description	release_year	age_certification	runtime
0	ts20945	The Three Stooges	SHOW	The Three Stooges were an American vaudeville ...	1934	TV-PG	19
1	tm19248	The General	MOVIE	During America's Civil War, Union spies steal ...	1926	NaN	78

In [53]:

```python
df2.head(2)
```

Out[53]:

	person_id	id	name	character	role
0	59401	ts20945	Joe Besser	Joe	ACTOR
1	31460	ts20945	Moe Howard	Moe	ACTOR

In [54]:

```python
# Shape of the dataset
print(df1.shape)
print(df2.shape)
```

```
(9871, 15)
(124235, 5)
```

In [55]:

```python
# Now let's combine these two datasets since they have common id.
df= pd.merge(df1,df2, on = "id")
```

In [56]:

```python
# Shows the columns of the data
df.columns
```

Out[56]:

```
Index(['id', 'title', 'type', 'description', 'release_year',
       'age_certification', 'runtime', 'genres', 'production_countrie
s',
       'seasons', 'imdb_id', 'imdb_score', 'imdb_votes', 'tmdb_popula
rity',
       'tmdb_score', 'person_id', 'name', 'character', 'role'],
      dtype='object')
```

In [57]:

```python
df.head(2)
```

Out[57]:

	id	title	type	description	release_year	age_certification	runtime
0	ts20945	The Three Stooges	SHOW	The Three Stooges were an American vaudeville ...	1934	TV-PG	19
1	ts20945	The Three Stooges	SHOW	The Three Stooges were an American vaudeville ...	1934	TV-PG	19

In [58]:

```python
# To find the duplicates
df.duplicated().sum()
```

Out[58]:

```
np.int64(168)
```

In [59]:

```python
df= df.drop_duplicates()
```

In [60]:

```python
df.duplicated().sum()
```

Out[60]:

```
np.int64(0)
```

In [61]:

```python
# Count the values of the type column of the dataset
df.value_counts(['type'])
```

Out[61]:

```
type
MOVIE     116026
SHOW        8153
Name: count, dtype: int64
```

Findings- This shows that we have total 8514 movies and 1357 shows in the dataset.

In [62]:

```python
#Counting of movies by genres'
df.value_counts(['genres'])
```

Out[62]:

```
genres
['drama']                                                 9853
['comedy']                                                5582
['drama', 'romance']                                      2512
['horror']                                                2448
['documentation']                                         2373
                                                          ...
['animation', 'family', 'scifi']                             1
['reality', 'music', 'documentation']                        1
['animation', 'fantasy', 'scifi', 'comedy', 'family']        1
['animation', 'reality', 'comedy', 'family']                 1
['drama', 'documentation', 'horror']                         1
Name: count, Length: 1965, dtype: int64
```

In [63]:

```python
# Information of the dataset.
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 124179 entries, 0 to 124346
Data columns (total 19 columns):
 #   Column                Non-Null Count   Dtype
---  ------                --------------   -----
 0   id                    124179 non-null  object
 1   title                 124179 non-null  object
 2   type                  124179 non-null  object
 3   description           124088 non-null  object
 4   release_year          124179 non-null  int64
 5   age_certification     56707 non-null   object
 6   runtime               124179 non-null  int64
 7   genres                124179 non-null  object
 8   production_countries  124179 non-null  object
 9   seasons               8153 non-null    float64
 10  imdb_id               118876 non-null  object
 11  imdb_score            118128 non-null  float64
 12  imdb_votes            118104 non-null  float64
 13  tmdb_popularity       124164 non-null  float64
 14  tmdb_score            114010 non-null  float64
 15  person_id             124179 non-null  int64
 16  name                  124179 non-null  object
 17  character             107902 non-null  object
 18  role                  124179 non-null  object
dtypes: float64(5), int64(3), object(11)
memory usage: 18.9+ MB
```

In [64]:

```python
# Statistical information about the dataset.
df.describe()
```

Out[64]:

	release_year	runtime	seasons	imdb_score	imdb_votes
count	124179.000000	124179.000000	8153.000000	118128.000000	1.181040e+05
mean	1996.307186	95.577706	2.337790	5.968327	2.327575e+04
std	27.752141	30.166205	3.130184	1.242262	8.846004e+04
min	1912.000000	1.000000	1.000000	1.100000	5.000000e+00
25%	1983.000000	83.000000	1.000000	5.200000	2.890000e+02
50%	2009.000000	93.000000	1.000000	6.100000	1.257000e+03
75%	2017.000000	109.000000	2.000000	6.800000	7.074000e+03
max	2022.000000	540.000000	49.000000	9.900000	1.133692e+06

In [65]:

```python
# Checking the null values of the dataset.
df.isnull().sum()
```

Out[65]:

```
id                           0
title                        0
type                         0
description                 91
release_year                 0
age_certification        67472
runtime                      0
genres                       0
production_countries         0
seasons                 116026
imdb_id                   5303
imdb_score                6051
imdb_votes                6075
tmdb_popularity             15
tmdb_score               10169
person_id                    0
name                         0
character                16277
role                         0
dtype: int64
```

**We can see a lot of missing values in the dataset. We have to take care of these null values. We have to remove some unnecessary columns, delete some rows, fill the values. **

In [66]:

```python
# drop seasons column
df.drop('seasons', axis=1, inplace = True)

# Drop the description column.
df.drop(['description'], axis= 1, inplace= True)

# drop imdb_id
df.drop('imdb_id', axis=1, inplace= True)

# drop id
df.drop('id', axis=1, inplace= True)
```

In [67]:

```python
df.head(2)
```

Out[67]:

	title	type	release_year	age_certification	runtime	genres
0	The Three Stooges	SHOW	1934	TV-PG	19	['comedy', 'family', 'animation', 'action', 'f...
1	The Three Stooges	SHOW	1934	TV-PG	19	['comedy', 'family', 'animation', 'action', 'f...

In [68]:

```python
df.isnull().sum()
```

Out[68]:

```
title                       0
type                        0
release_year                0
age_certification       67472
runtime                     0
genres                      0
production_countries        0
imdb_score               6051
imdb_votes               6075
tmdb_popularity            15
tmdb_score              10169
person_id                   0
name                        0
character               16277
role                        0
dtype: int64
```

In [69]:

```python
# Renaming some columns
df= df.rename(columns= {'age_certification' : 'rating',
                        'production_countries': 'countries'
})
```

In [70]:

```python
# Filling missing values on charector and rating column
df['character'].fillna('unavailable', inplace= True)
df['rating'].fillna('Unrated', inplace= True)
```

In [71]:

```python
# Insert missing values on imdb_score, imdb_votes and tmdb+score
mean_value= df['imdb_score'].mean()
df['imdb_score'].fillna(mean_value, inplace= True)

mean_value= df['tmdb_score'].mean()
df['tmdb_score'].fillna(mean_value, inplace= True)

mean_value= df['imdb_votes'].mean()
df['imdb_votes'].fillna(mean_value, inplace= True)
```

In [72]:

```python
df.isnull().sum()
```

Out[72]:

```
title               0
type                0
release_year        0
rating              0
runtime             0
genres              0
countries           0
imdb_score          0
imdb_votes          0
tmdb_popularity    15
tmdb_score          0
person_id           0
name                0
character           0
role                0
dtype: int64
```

In [73]:

```python
# Now, let's drop remaing null values
df= df.dropna()
```

In [74]:

```python
df.isnull().sum()
```

Out[74]:

```
title              0
type               0
release_year       0
rating             0
runtime            0
genres             0
countries          0
imdb_score         0
imdb_votes         0
tmdb_popularity    0
tmdb_score         0
person_id          0
name               0
character          0
role               0
dtype: int64
```

In [75]:

```python
pd.crosstab(df["rating"], df["type"])
```

Out[75]:

type	MOVIE	SHOW
rating		
G	3839	0
NC-17	116	0
PG	9617	0
PG-13	12492	0
R	25931	0
TV-14	0	1583
TV-G	0	242
TV-MA	0	1732
TV-PG	0	501
TV-Y	0	298
TV-Y7	0	356
Unrated	64018	3439

In [76]:

```python
pd.crosstab(df["countries"], df["type"])
```

Out[76]:

type	MOVIE	SHOW
countries		
['AF', 'US']	17	0
['AF']	1	0
['AL', 'CY', 'GR']	9	0
['AN', 'US']	9	0
['AR', 'US', 'AQ']	3	0
...	...	...
['ZA', 'LY']	2	0
['ZA', 'US', 'NZ', 'CA']	92	0
['ZA', 'US']	16	0
['ZA']	167	0
[]	2150	136

482 rows × 2 columns

In [77]:

```python
# Unique rating values.
df["rating"].unique()
```

Out[77]:

```
array(['TV-PG', 'Unrated', 'PG', 'G', 'PG-13', 'R', 'TV-G', 'TV-Y',
       'TV-14', 'NC-17', 'TV-Y7', 'TV-MA'], dtype=object)
```

In [78]:

```python
df["genres"].value_counts()
```

Out[78]:

```
genres
['drama']                                                9850
['comedy']                                               5580
['drama', 'romance']                                     2512
['horror']                                               2441
['documentation']                                        2373
                                                         ...
['scifi', 'documentation']                                  1
['drama', 'documentation', 'horror']                        1
['comedy', 'history', 'war', 'documentation']               1
['animation', 'reality', 'comedy', 'family']                1
['sport', 'documentation', 'reality']                       1
Name: count, Length: 1964, dtype: int64
```

In [79]:

```python
# let's explode the dataframe to have one row per genre.
df_exploded= df.explode('genres')

genre_counts= df_exploded['genres'].value_counts()
print(genre_counts)
```

```
genres
['drama']                                                9850
['comedy']                                               5580
['drama', 'romance']                                     2512
['horror']                                               2441
['documentation']                                        2373
                                                         ...
['scifi', 'documentation']                                  1
['drama', 'documentation', 'horror']                        1
['comedy', 'history', 'war', 'documentation']               1
['animation', 'reality', 'comedy', 'family']                1
['sport', 'documentation', 'reality']                       1
Name: count, Length: 1964, dtype: int64
```

In [80]:

```python
# Top 10 genres
top_10_genre= df['genres'].value_counts().to_frame()
top_10_genre.head(10)
```

Out[80]:

	count
genres	
['drama']	9850
['comedy']	5580
['drama', 'romance']	2512
['horror']	2441
['documentation']	2373
['drama', 'comedy']	1991
['comedy', 'drama']	1986
['thriller']	1848
['romance', 'comedy']	1622
['comedy', 'romance']	1608

In [81]:

```python
# Top 10 countries with most films/shows on amazon prime
top_10_countries= df['countries'].value_counts().to_frame()
top_10_countries.head(10)
```

Out[81]:

	count
countries	
['US']	69216
['IN']	11174
['GB']	7204
['CA']	3200
[]	2286
['JP']	2114
['AU']	1591
['FR']	1490
['CN']	1238
['GB', 'US']	1229

In [82]:

```python
# Top 10 rating film/shows
top_10_rated= df['rating'].value_counts().to_frame()
top_10_rated.head(10)
```

Out[82]:

	count
rating	
Unrated	67457
R	25931
PG-13	12492
PG	9617
G	3839
TV-MA	1732
TV-14	1583
TV-PG	501
TV-Y7	356
TV-Y	298

In [83]:

```python
# Count number of movies with release year.
release_year_counts= df['release_year']. value_counts().to_frame()
release_year_counts
```

Out[83]:

	count
release_year	
2019	8455
2021	7501
2020	6923
2018	6917
2017	6296
...	...
1922	44
1914	33
1928	28
1912	11
1917	8

110 rows × 1 columns

In [84]:

```python
# top 10 actors with the most films/shows.
filtered_actors= df[df['name'] != 'unavailable']
top_actors= filtered_actors['name'].value_counts().head(10).to_frame()
top_actors.head(10)
```

Out[84]:

	count
name	
George 'Gabby' Hayes	49
Roy Rogers	45
Bess Flowers	44
Joseph Kane	41
Gene Autry	40
Sam Newfield	38
Nassar	37
Charles King	35
Herman Hack	35
George Morrell	34

In [85]:

```python
df.value_counts('role')
```

Out[85]:

```
role
ACTOR       115784
DIRECTOR      8380
Name: count, dtype: int64
```

In [86]:

```python
#Most Popular movie on amazon prime based on imdb rating.
movies_df= df[df['type'].str.lower()=='movie']
movies_df= movies_df.drop_duplicates(subset= 'title')
top_movies= movies_df.sort_values(by= 'imdb_score', ascending= False)
print(top_movies[['title','imdb_score']].head(10))
```

```
                                           title  imdb_score
119143                                Pawankhind         9.9
121928                                Tari Sathe         9.4
118060                                  Jai Bhim         9.3
107116                          Soorarai Pottru          9.2
110301      Alexander Babu: Alex in Wonderland           9.2
109096                                     Quota         9.2
114751                             Toxification          9.1
34981   Steven Banks: Home Entertainment Center          9.1
110462                                   Peranbu         9.1
119789                                     #Home         9.0
```

In [87]:

```python
#Most Popular movie on amazon prime based on tmdb rating.

movies_df= df[df['type'].str.lower()=='movie']
movies_df= movies_df.drop_duplicates(subset= 'title')
top_movies= movies_df.sort_values(by= 'tmdb_score', ascending= False)
print(top_movies[['title','tmdb_score']].head(10))
```

```
                                           title  tmdb_score
112832   Jonas Kaufmann - Ein Weltstar ganz privat        10.0
118388                                 Skinwalker         10.0
37490     COINTELPRO: The FBI's War on Black America      10.0
97150                          Nurses Do It Better        10.0
98549                Rise Of The Great White Shark        10.0
98643          Alan Jackson: Small Town Southern Man      10.0
122328                              Call Time The Finale   10.0
122173    The Diabolical Schemes of Thadeus Jackson       10.0
99378                                 Human Comedy        10.0
94693                                  Unspeakable        10.0
```

In [88]:

```python
#Most Popular show on amazon prime based on tmdb rating.
shows_df= df[df['type'].str.lower()=='show']
shows_df= shows_df.drop_duplicates(subset= 'title')
top_shows= shows_df.sort_values(by= 'tmdb_score', ascending= False)
print(top_shows[['title','tmdb_score']].head(10))
```

```
                              title  tmdb_score
105759                      Morphle        10.0
110421  Wild and Free: Florianópolis       10.0
72540          Great Canal Journeys        10.0
63300           Cosmic Quantum Ray         10.0
62634          Paula's Home Cooking        10.0
116113                  61st Street        10.0
60884            The Future Is Wild        10.0
72980         Secrets of the Castle        10.0
115673             Flying Tiger II         10.0
60394               The Invisibles         10.0
```

In [89]:

```python
df.groupby("genres")["runtime"].mean().sort_values(ascending= False)
```

Out[89]:

```
genres
['western', 'history', 'romance', 'war']                    328.0
['drama', 'documentation', 'history']                       260.0
['crime', 'drama', 'family', 'scifi', 'thriller']           258.0
['drama', 'european', 'history']                            246.0
['action', 'romance', 'family']                             228.0
                                                             ...
['comedy', 'romance', 'drama', 'european']                   10.0
['animation', 'action']                                       5.0
['drama', 'animation', 'action', 'scifi']                     2.0
['crime', 'thriller', 'romance']                              1.0
['comedy', 'animation', 'family', 'european']                 1.0
Name: runtime, Length: 1964, dtype: float64
```

In [90]:

```python
# Top 10 genres.
top_10_genres= genre_counts.head(10)
```

In [91]:

```python
plot_data= top_10_genres.reset_index()
plot_data.columns= ['genres', 'count']
plot_data.head(2)
```

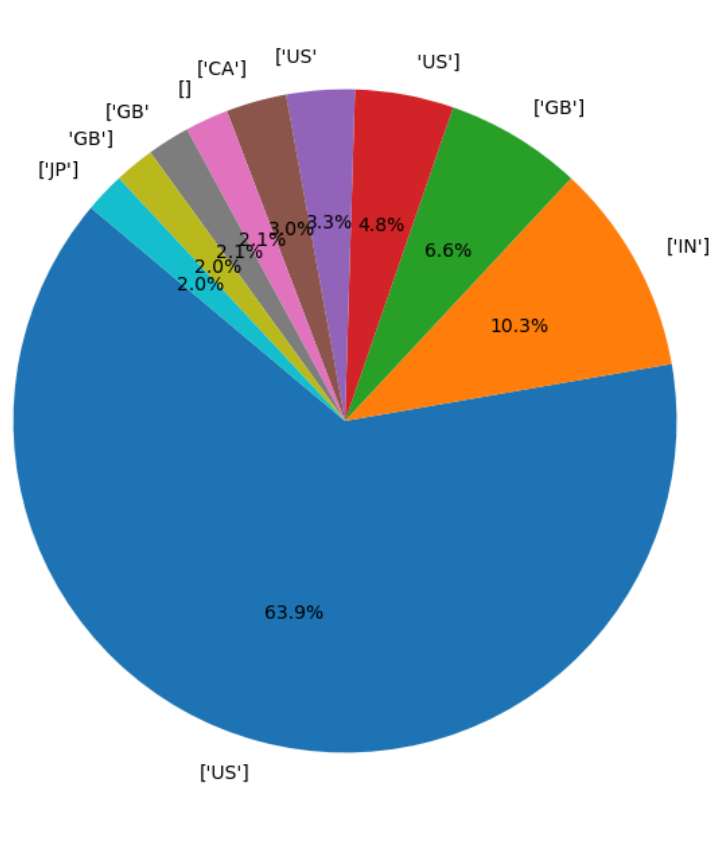
Out[91]:

	genres	count
0	['drama']	9850
1	['comedy']	5580

Section 2: Visualization.

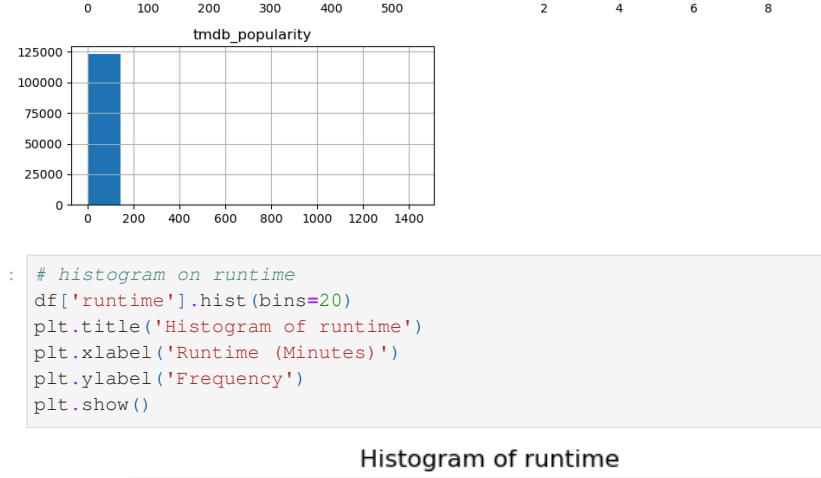
In [92]:

```python
genre_counts= ( df["genres"].dropna()
.str.split(",")
.explode()
.value_counts()
.head(10))
# let's create pie chart to show top 10 genres on Amazon prime.

plt.figure(figsize=(8,8))
plt.pie(
    genre_counts,
    labels= genre_counts.index,
    autopct= "%1.1f%%",
    startangle=140)
plt.title(" TOP 10 GENRES ON A PRIME")
plt.show()
```



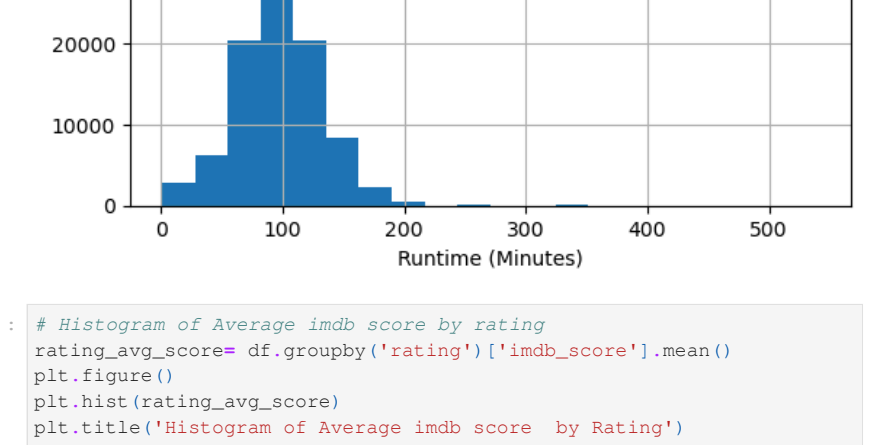
In [93]:

```python
# Content distribution across different regions.
# Top 10 regions
region_counts= (
    df["countries"]
    .str.split(",")
    .explode()
    .value_counts().head(10)

)
plt.figure(figsize=(8,8))
plt.pie(
    region_counts,
    labels= region_counts.index,
    autopct= "%1.1f%%",
    startangle=140)
plt.title(" TOP 10 Regional's in an AMAZON PRIME")
plt.show()
```



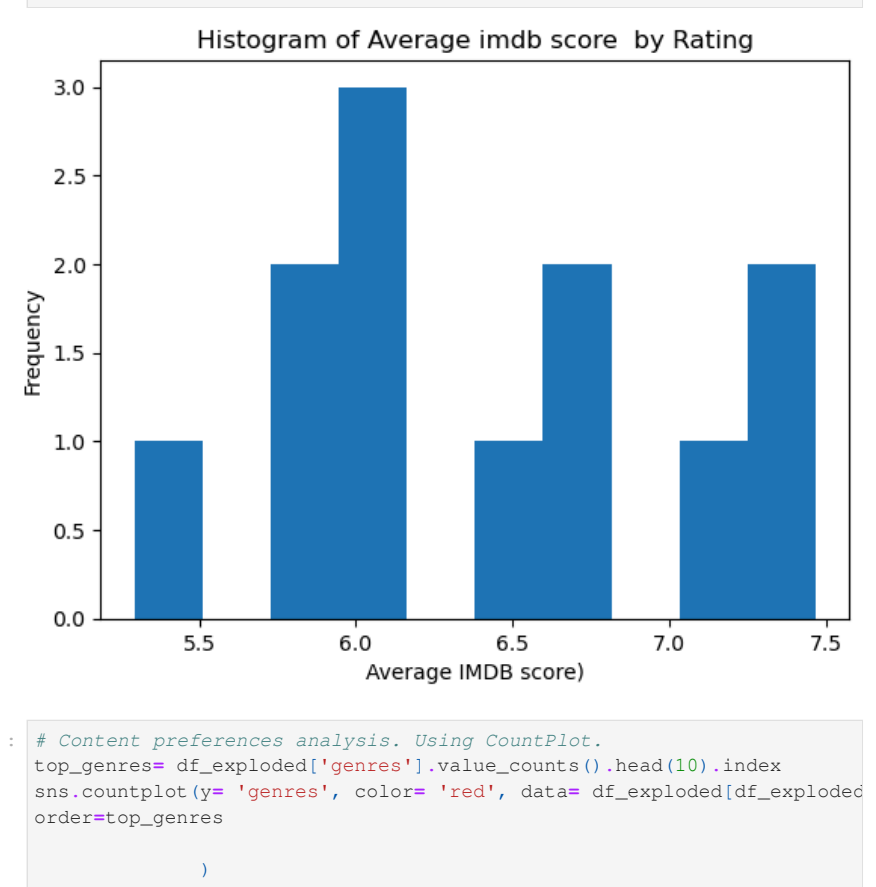
In [94]:

```python
# Histogram to see imdb_score, runtime, tmdb-popularity.
df.hist(column= ['runtime', 'imdb_score', 'tmdb_popularity'], figsize=
plt.tight_layout()
plt.show()
```



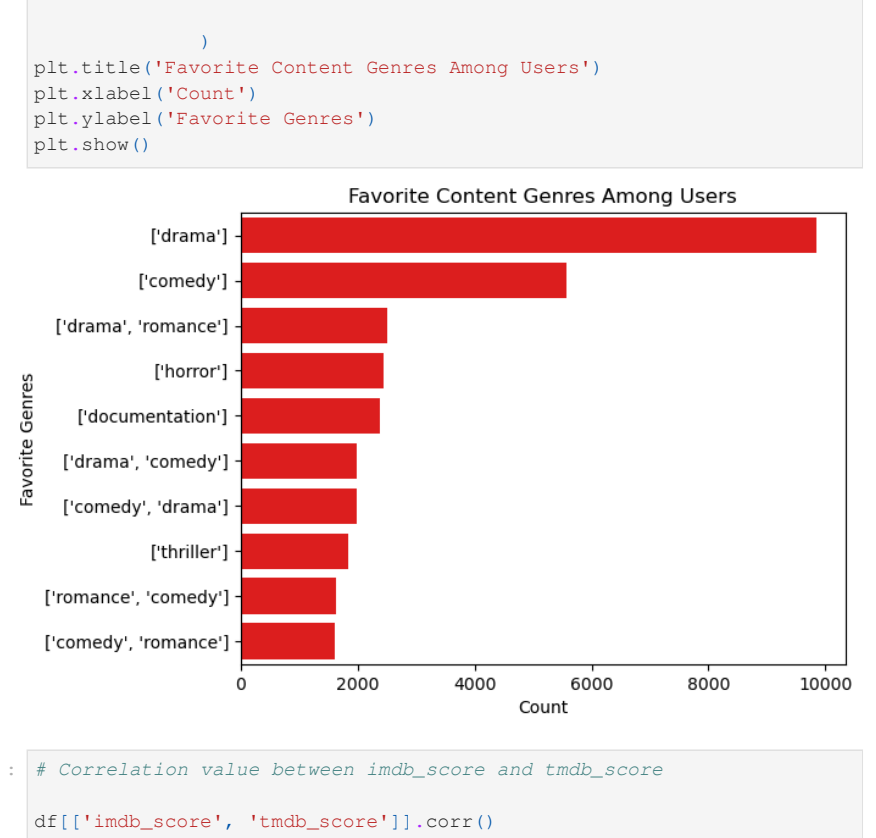
In [95]:

```python
# histogram on runtime
df['runtime'].hist(bins=20)
plt.title('Histogram of runtime')
plt.xlabel('Runtime (Minutes)')
plt.ylabel('Frequency')
plt.show()
```



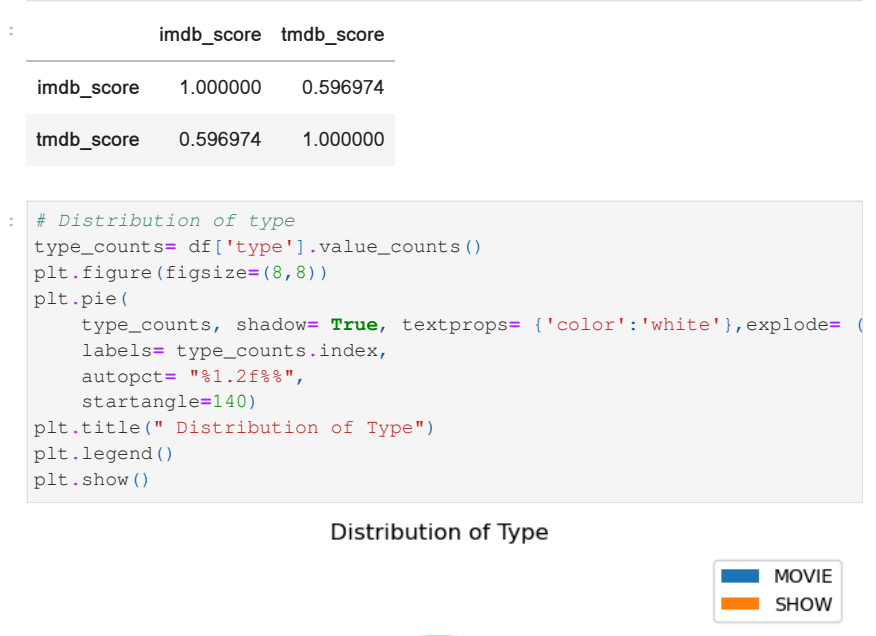
In [96]:

```python
# Histogram of Average imdb score by rating
rating_avg_score= df.groupby('rating')['imdb_score'].mean()
plt.figure()
plt.hist(rating_avg_score)
plt.title('Histogram of Average imdb score  by Rating')
plt.xlabel('Average IMDB score)')
plt.ylabel('Frequency')
plt.show()
```



In [97]:

```python
# Content preferences analysis. Using CountPlot.
top_genres= df_exploded['genres'].value_counts().head(10).index
sns.countplot(y= 'genres', color= 'red', data= df_exploded[df_exploded
order=top_genres

             )
plt.title('Favorite Content Genres Among Users')
plt.xlabel('Count')
plt.ylabel('Favorite Genres')
plt.show()
```



In [98]:

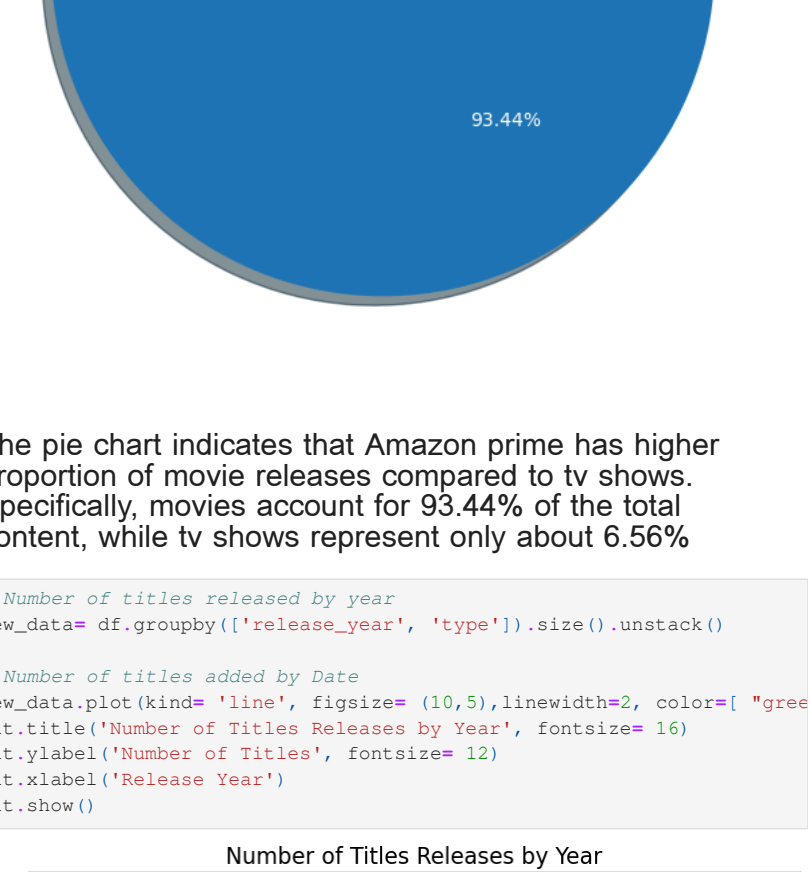
```python
# Correlation value between imdb_score and tmdb_score

df[['imdb_score', 'tmdb_score']].corr()
```

Out[98]:

	imdb_score	tmdb_score
imdb_score	1.000000	0.596974
tmdb_score	0.596974	1.000000

In [99]:

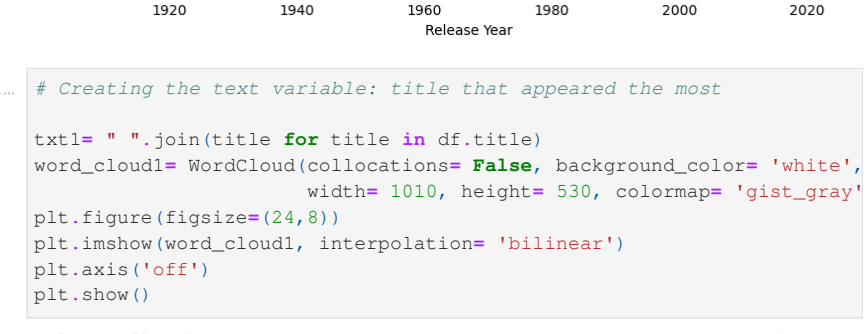
```python
# Distribution of type
type_counts= df['type'].value_counts()
plt.figure(figsize=(8,8))
plt.pie(
    type_counts, shadow= True, textprops= {'color':'white'},explode= (
    labels= type_counts.index,
    autopct= "%1.2f%%",
    startangle=140)
plt.title(" Distribution of Type")
plt.legend()
plt.show()
```



The pie chart indicates that Amazon prime has higher proportion of movie releases compared to tv shows. Specifically, movies account for 93.44% of the total content, while tv shows represent only about 6.56%

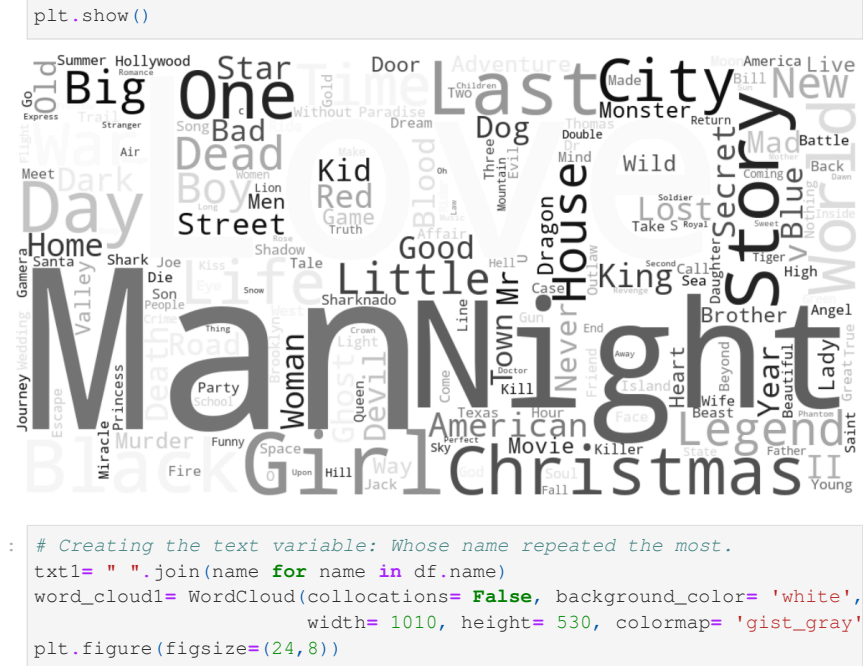
In [100…

```python
# Number of titles released by year
new_data= df.groupby(['release_year', 'type']).size().unstack()

# Number of titles added by Date
new_data.plot(kind= 'line', figsize= (10,5),linewidth=2, color=[ "gree
plt.title('Number of Titles Releases by Year', fontsize= 16)
plt.ylabel('Number of Titles', fontsize= 12)
plt.xlabel('Release Year')
plt.show()
```



In [101…

```python
# Creating the text variable: title that appeared the most

txt1= " ".join(title for title in df.title)
word_cloud1= WordCloud(collocations= False, background_color= 'white',
                       width= 1010, height= 530, colormap= 'gist_gray'
plt.figure(figsize=(24,8))
plt.imshow(word_cloud1, interpolation= 'bilinear')
plt.axis('off')
plt.show()
```

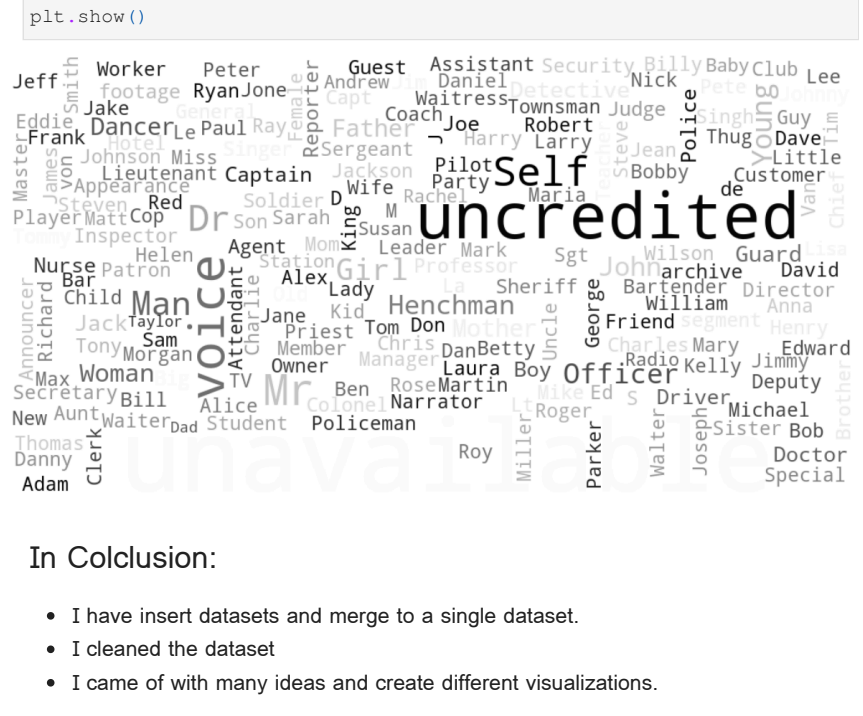


In [ ]:

```python
# Creating the text variable: Whose name repeated the most.
txt1= " ".join(name for name in df.name)
word_cloud1= WordCloud(collocations= False, background_color= 'white',
                       width= 1010, height= 530, colormap= 'gist_gray'
plt.figure(figsize=(24,8))
plt.imshow(word_cloud1, interpolation= 'bilinear')
plt.axis('off')
plt.show()
```

In [102…

```python
# Creating the text variable: charecter that appeared the most.
txt1= " ".join(character for character in df.character)
word_cloud1= WordCloud(collocations= False, background_color= 'white',
                       width= 1010, height= 530, colormap= 'gist_gray'
plt.figure(figsize=(24,8))
plt.imshow(word_cloud1, interpolation= 'bilinear')
plt.axis('off')
plt.show()
```



In Colclusion:

- I have insert datasets and merge to a single dataset.
- I cleaned the dataset
- I came of with many ideas and create different visualizations.

Insights from the research.

- Movies usually make a larger share than TV shows.
- Most popular shows span multiple seasons.
- Higher IMDb scores usually align with higher TMDb score
- US, India, Uk are top countries that produces movies/shows
- Number of moveies have significantly increased after 2020.
- the word "Love" appeared higher in the movies/tv shows.